

Práctica 6. Excepciones, triggers y constraints

Profesor

David Guijo Rubio

Email

dguijo@uco.es



Índice

- Excepciones
- Constraints
- Triggers

David Guijo Rubio



Índice

- **Excepciones**
- Constraints
- Triggers

David Guijo Rubio



Excepciones

- Es la tercera sección de un bloque de PL/SQL. Se encarga de un control de errores adicional, para evitar anomalías en un código.
- Un ejemplo clásico de excepción es una división entre 0.
- El objetivo es capturar la situación concreta para modificar su comportamiento.
- Cuando se produce la situación concreta se ejecuta el bloque **EXCEPTION**.

Excepciones

```
set serveroutput on;  
  
DECLARE  
    VALOR_NEGATIVO EXCEPTION;  
    valor NUMBER := -1;  
  
BEGIN  
    IF valor < 0 THEN  
        RAISE VALOR_NEGATIVO;  
    END IF;  
  
EXCEPTION  
    WHEN VALOR_NEGATIVO THEN  
        dbms_output.put_line('El valor no puede ser negativo');  
  
END;
```

David Guijo Rubio



Excepciones: manejadores de error

- Se pueden enviar errores personalizados cuando se produce una excepción.

Función: `RAISE_APPLICATION_ERROR (<número entre -20001 y -20999>, <mensaje>);`

DECLARE

`v_div NUMBER;`

BEGIN

`v_div:= 1/0;`

EXCEPTION

`WHEN OTHERS THEN RAISE_APPLICATION_ERROR(-20001,'No se puede dividir por cero');`

END;

NOTA: OTHERS representa cualquier condición de excepción que no ha sido declarada previamente.

Índice

- Exceptions
- **Constraints**
- Triggers

Constraints

- Son reglas automáticas que garantizan la integridad de la base de datos, es decir, que cumplan un criterio.
- Se ejecutan sin necesidad de hacer un bloque específico de PL/SQL.
- Se crean durante un CREATE TABLE o un ALTER TABLE. Tipos:
 - NOT NULL
 - UNIQUE
 - PRIMARY KEY
 - FOREIGN KEY
 - CHECK

Constraints

- Ejemplo:

```
CREATE TABLE empleados (  
    id_empleado NUMBER PRIMARY KEY,  
    nombrecompleto VARCHAR2(50) NOT NULL,  
    salario NUMBER CHECK (salario >= 0)  
);
```

Creamos una tabla con los votantes que están en situación activa. Incluimos 3 atributos, cada uno con una restricción.

Constraints

- Se pueden definir con un nombre específico:

```
CREATE TABLE empleados (  
    id_empleado NUMBER PRIMARY KEY,  
    nombrecompleto VARCHAR2(50) NOT NULL,  
    salario NUMBER CONSTRAINT salario_positivo CHECK (salario >= 0)  
);
```

Le damos un nombre específico a la constraint, en este caso “salario_positivo”.

¿Qué pasa si añadimos alguien con salario de -1?

Constraints

- Se puede añadir una restricción a una tabla ya existente:

```
ALTER TABLE empleados
```

```
ADD CONSTRAINT salario_positivo CHECK (salario >= 0);
```

Aquí estamos añadiendo una restricción a un atributo de una tabla YA existente.

Constraints

- De igual forma, se puede eliminar una restricción a una tabla ya existente:

```
ALTER TABLE empleados
```

```
DROP CONSTRAINT salario_positivo;
```

Aquí estamos eliminado una restricción a un atributo de una tabla YA existente.

Constraints

- Para evitar añadir y eliminar constraints constantemente, estas se pueden deshabilitar o habilitar:

ALTER TABLE empleados

[DISABLE | ENABLE] CONSTRAINT salario_positivo;

Índice

- Excepciones
- Constraints
- **Triggers**

Triggers

Un trigger es un bloque PL/SQL con un nombre asignado en Oracle Database, el mismo se ejecuta automáticamente cuando un evento en específico ocurre.

El evento puede ser de estos tipos:

- **Lenguaje de manipulación de datos.** Cuando en una tabla se ejecuta INSERT, UPDATE, o DELETE. Por ejemplo, si se define un trigger que se ejecuta antes que INSERT, se ejecutará antes que sea insertada una nueva fila.
- **Lenguaje de definición de datos.** Cuando se ejecuta CREATE o ALTER. Estos son usados usualmente para auditar cambios en los campos de la tabla.
- **Eventos del sistema** como inicio o apagado de la base de datos.
- **Eventos de conexión** tales como *login* o *logout*.

Triggers

- Estructura general:

```
CREATE {OR REPLACE} TRIGGER nombre_trigger
    [BEFORE | AFTER]
    [INSERT | DELETE | UPDATE {OF columnas}] [ OR [INSERT | DELETE | UPDATE {OF columnas}]...]
    ON tabla
    [FOR EACH ROW [WHEN condicion disparo]]
[DECLARE]
    -- Declaración de variables locales
BEGIN
    -- Instrucciones de ejecución
[EXCEPTION]
    -- Instrucciones de excepción
END;
```

Nota: si borramos la tabla sobre la que tenemos un trigger, se borra el trigger también.

David Guijo Rubio



Triggers

- Un uso común de los triggers es el de auditoría: comprobar quién y cuándo se ha hecho una operación determinada. Para ello, se tiene por costumbre crear otra tabla donde se añada esta información. Como se desconoce el id con el que vamos a insertar, podemos utilizar un timestamp de la fecha (aunque no es del todo correcto, lo apropiado sería generar una secuencia con incremento 1):

```
CREATE TABLE comprobacion_empleados (  
    fecha      TIMESTAMP PRIMARY KEY,  
    usuario    VARCHAR2(30),  
    operacion  VARCHAR2(10),  
    id_empleado NUMBER  
);
```

Triggers: predicados condicionales

- Contamos con los siguientes:
- Inserting: es TRUE cuando se ejecuta una orden de insert.
- Deleting: es TRUE cuando se ejecuta una orden de delete.
- Updating: es TRUE cuando se ejecuta una orden de update.

Ejemplo:

```
CREATE OR REPLACE TRIGGER comprobacion_trigger BEFORE INSERT OR DELETE ON empleados FOR EACH ROW
```

```
BEGIN
```

```
    IF INSERTING THEN
```

```
        INSERT INTO comprobacion_empleados(fecha, usuario, operacion, id_empleado) VALUES (SYSTIMESTAMP, USER, 'INSERT', :NEW.id_empleado);
```

```
    ELSIF DELETING THEN
```

```
        INSERT INTO comprobacion_empleados(fecha, usuario, operacion, id_empleado) VALUES (SYSTIMESTAMP, USER, 'DELETE', :OLD.id_empleado);
```

```
    END IF;
```

```
END;
```

David Guijo Rubio



Triggers: *:new* y *:old*

- Para acceder a los registros que están siendo añadidos o eliminados debemos hacer uso de *:new* y *:old*:
- *:new* → valores a ser insertados o actualizados.
- *:old* → valores originales a ser actualizados o eliminados.

Nota: *:new* no está definido cuando se hace un DELETE. *:old* tampoco en un INSERT.

Triggers

- Comprobemos como funciona:

```
INSERT INTO empleados VALUES (1, 'Paco Martínez García', 1800);  
SELECT * FROM comprobacion_empleados;
```

Debemos obtener algo como:

FECHA	USUARIO	OPERACION	ID_EMPLEADO

09/12/25 11:48:25,381345000	I22GURUD	INSERT	1

David Guijo Rubio



Triggers: cláusula *when*

- Se utiliza para restringir triggers a nivel de fila, dependiendo de una condición:

```
CREATE OR REPLACE TRIGGER comprobacion_trigger BEFORE INSERT OR DELETE ON empleados  
FOR EACH ROW WHEN (new.salario > 1500)
```

```
BEGIN
```

```
IF INSERTING THEN
```

```
INSERT INTO comprobacion_empleados(fecha, usuario, operacion, id_empleado) VALUES (SYSTIMESTAMP, USER, 'INSERT', :NEW.id_empleado);
```

```
ELSIF DELETING THEN
```

```
INSERT INTO comprobacion_empleados(fecha, usuario, operacion, id_empleado) VALUES (SYSTIMESTAMP, USER, 'DELETE', :OLD.id_empleado);
```

```
END IF;
```

```
END;
```

En este caso solo se dispara el trigger si el salario está por encima de los 1500 euros.